CAD for VLSI Design

2025 ICCAD Contest – Check Point 3

Problem E : RTL Re-Connection

Problem Report

Advisor: Yu-Guang Chen

Student :電機系/所

113521142 沈立揚

110501521 徐松廷

110501022 陳冠宏

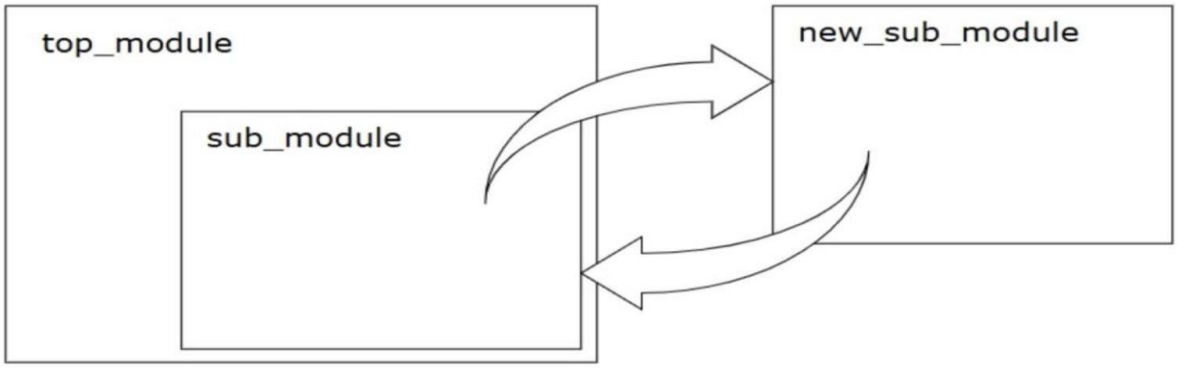
110501516 蔡昀翰

Content

Part 1. Purpose of the Problem 2
1. Introduction 2
2. Problem Statement 2
Part 2. Algorithm and Program Structure 3
1. Parser and Output File Logic 3
A. Parser Data Structure 3
B. Read Sub Module 3
C. Read New Sub Module 4
D. Output File Logic 5
2. Signal Permutation and AST Comparison 5
A. Purposes 5
B. Main Idea of Signal Permutation and AST Comparison 5
C. Flow Chart of Programs 6
D. Data Structure- Abstract Syntax Tree 6
E. Algorithm- Building Abstract Syntax Tree 7
F. Algorithm- Signal Pairing Permutation 7
G. Algorithm- Normalizing Abstract Syntax Tree 8
H. Algorithm- Signal Matching 9
3. RTL to Gate-Level Netlist 9
A. Purposes 9
B. Updated Flow Chart 9
C. Arithmetic operations/conditional statements to logic gate netlist 9
4. CNF & SAT Solver 11
A. Conjunctive Normal Form 11
B. SAT Solver 12
Part3. Results on Public Testcases 13
1. Public TestCase 0 13
2. Public TestCase 1 13
3. Self-Generated TestCase 2 14
4. Self-Generated TestCase 3 15
5. Public case2 and 4 parser 16
6. Conclusion 16
Part 4. Compilation and Execution 17
1. AST_comparison_case_0_1 17
2. Case_2_and_4_Parser 17
Part 5. Conclusions 18

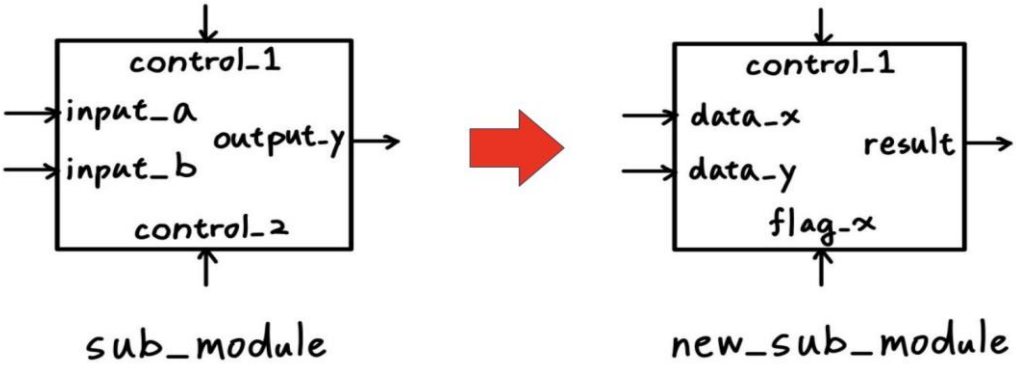
Part 1. Purpose of the Problem

1. Introduction

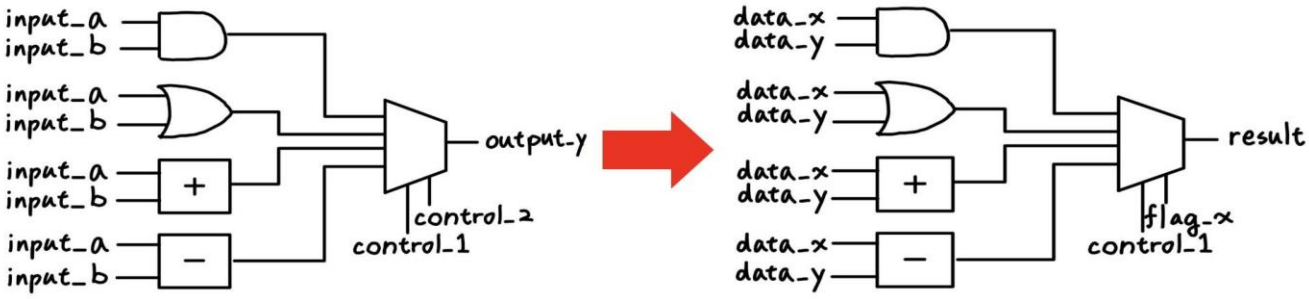


首先要介紹的是題目背景，RTL 設計是指用 verilog 等硬體語言來設計電路的步驟，因為他會很大程度的影響電路的功耗及面積，所以在 ic design flow 裡面是一個很重要的步驟。而在 RTL 設計中常常用 top module 來整合像是第三方 IP 或自定義模組等 sub module ，我們透過這樣來組成完整的系統，但在 sub module 在進行版本更新後，訊號的名稱還有 port 或是 function 可能都會有改變，所以我們要將 top module 的 port 重新拉線，之後再驗證並確保重新拉完線後的 function 和 logic 還是跟原本一樣。

2. Problem Statement



接著要介紹的是這題的詳細規則，在我們更換 sub module 後，我們需要考慮三個問題，首先是名稱更變的問題，在 sub module 更新之後，我們的訊號名稱會有改變，所以要找到新舊名稱的對應關係，像是上圖中 input a 會對應 data x，而 output y 則會對應 result。



而如果我們沒辦法直接看出 input output 訊號的對應關係的話，那我們就會透過比對 logic circuit 來找到他們的對應關係，像是上圖中我們可以從 and gate input 來得知 input a 跟 input b 是對應 data x 跟 data y，而最後我們則要確認在 sub module 更新以後，他的 output logic 會跟之前的一樣。

Part 2. Algorithm and Program Structure

1. Parser and Output File Logic

A. Parser Data Structure

module_info

string	module_name		
vector<string>	input1	· · ·	input n
vector<string>	output1	· · ·	output n
vector<string>	wire1	· · ·	wire n
vector<gate_info>	gate 1	· · ·	gate n

首先要介紹的是我們 parser 是如何讀取還有儲存 input 資料。我們會先用 struct 存取 parser 讀到的 gate 以及 module 的資料，而我們 module 裡面會儲存他的名字，還有他的所有 input output wire、gate 跟 gate 之間的 wire，以及 module 裡面的所有 gate。

gate_info

string	gate_name		
vector<string>	input1	· · ·	input n
string	output		

而我們每個 gate 都會存他的 gate 名稱，像是 and 或是 or 之類的，還有這個 gate 所有跟他連接的 input output wire 名稱。

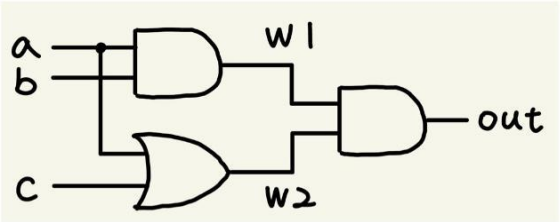
B. Read Sub Module

```
module sub_module1{
    input wire a,
    input wire b,
    input wire c,
    output wire out
};
wire w1;
wire w2;
```

sub_module[0]

string	sub_module1		
vector<string>	a	b	c
vector<string>	out		
vector<string>	w1	w2	
vector<gate_info>	gate[0]	gate[1]	gate[2]

我們的 parser 一開始會讀取舊 sub module 的 input output 還有 wire 的資料，以上圖為例，我們讀到了 sub module 1 後，我們就會把他存在 vector sub module 的第一格，所以這樣就能存放多個 module，而裡面我們會分別儲存 input a b c 還有 output out 跟 wire w1 w2。



gate[2]

string	and	
vector<string>	w1	w2
string	out	

gate[1]

string	or	
vector<string>	a	c
string	w2	

gate[0]

string	and	
vector<string>	a	b
string	w1	

而 sub module 裡面的邏輯閘則會存在 vector gate 裡面，以上圖為例，我們會把這個 and gate 還有他的 input a b 跟 output w1 存在 gate 的第一格，這樣我們就能存 module 裡面的所有 gate。

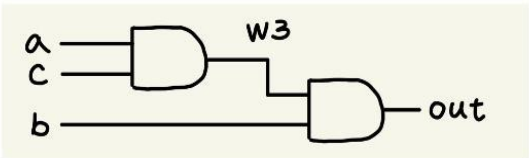
C. Read New Sub Module

```
module new_sub_module1{
    input wire a,
    input wire c,
    input wire b,
    output wire out
};
wire w3;
```

new_sub_module[0]

string	new_sub_module		
vector<string>	a	c	b
vector<string>	out		
vector<string>	w3		
vector<gate_info>	gate[0]	gate[1]	

在讀取新的 sub module 後我們一樣會把 input output 還有 wire 存到一個 vector new sub module 裡面，這時候讀到的 input output 順序還有 gate 種類個數可能會跟舊的不一樣，但這沒關係，我們之後會進行資料重新匹配，這部分會留到 part b 那裡細講。



gate[1]

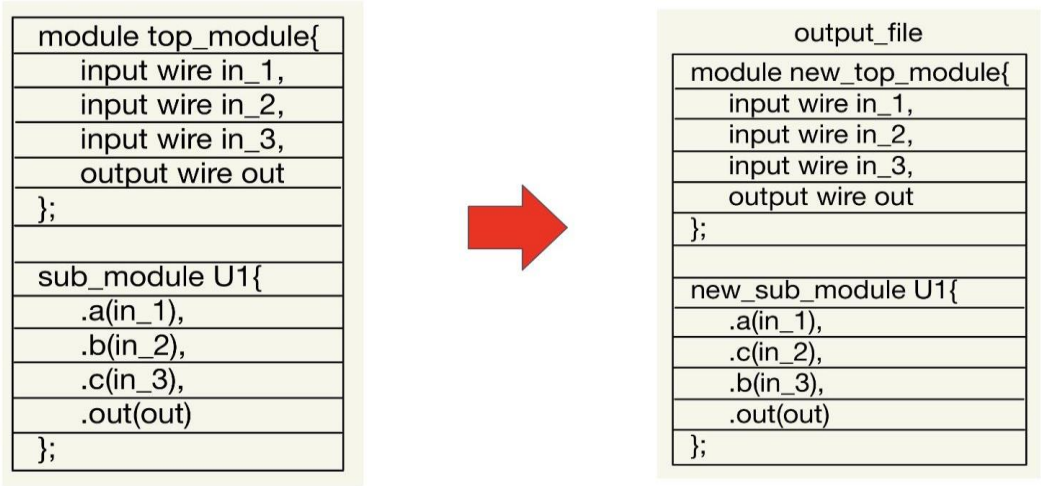
string	and	
vector<string>	a	c
string	w3	

gate[0]

string	and	
vector<string>	w3	b
string	out	

我們讀到新 sub module 的邏輯閘時跟前面一樣，會把它們存在 vector gate 裡面，這裡的 gate 還有接線跟舊的 sub module 不太一樣，所以我們比對完後會把正確的 input output 輸出。

D. Output File Logic



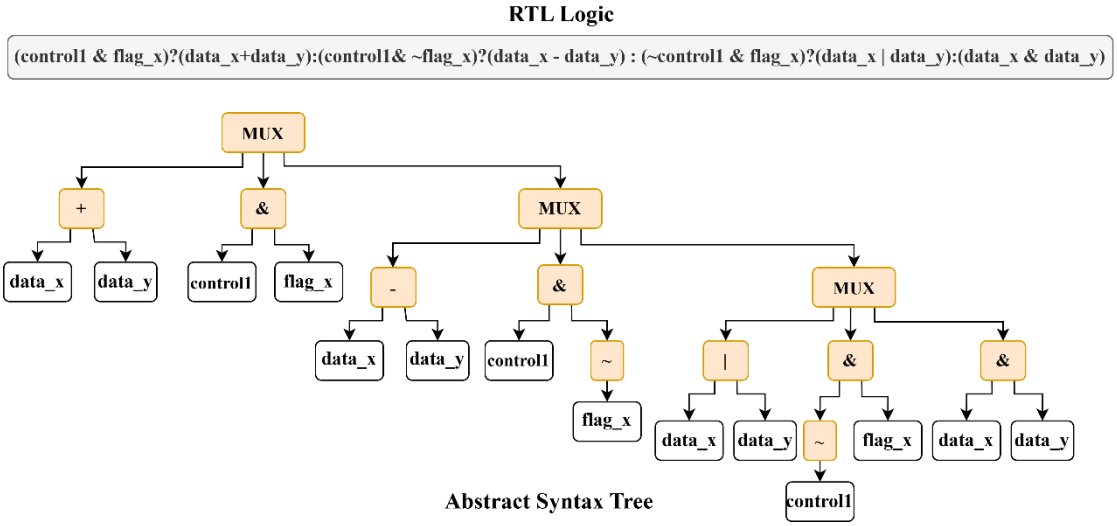
一開始我們讀取 top module 檔的時候會把每行都存到 vector 裡面，最後要輸出時我們再 go through 整個 vector，把要改變的 input output 訊號蓋過去。像是上圖因為邏輯改變了所以 input 訊號順序不一樣，所以我們在新的 sub module 檔就會改成新的 input，然後再作為 output 檔輸出出去。

2. Signal Permutation and AST Comparison

A. Purposes

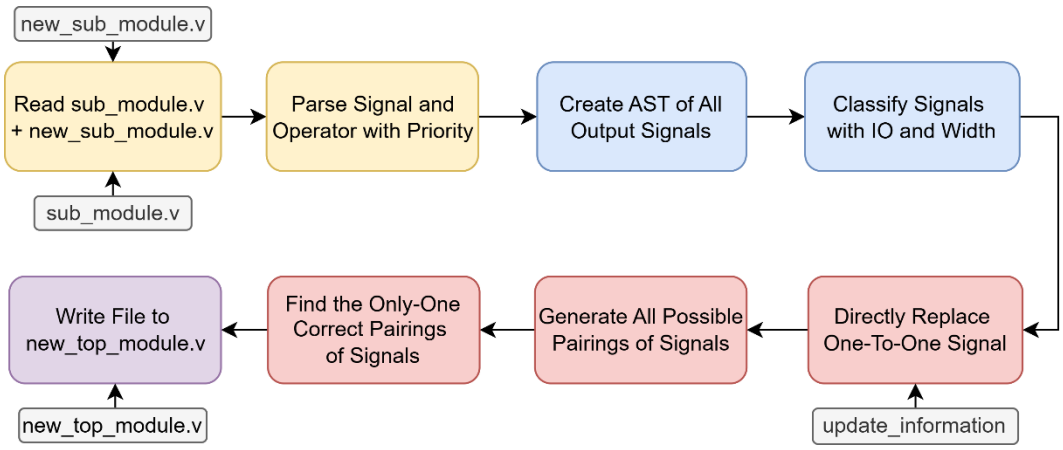
針對 Problem E，我們將解題策略劃分為兩個層次： Simple Test Case 與進階測資 Advanced Test Case。在簡單測資部分，我們不進行邏輯化簡，僅針對訊號進行結構性比對，以提升整體處理效率並節省執行時間。相較之下，進階測資則納入邏輯層級的分析與 gate-level 結構的處理，因此需採用較為複雜的演算法，以確保比對結果的準確性與完整性。藉由將兩者分別實作，我們能夠在效能與精度之間取得良好平衡，進而提升整體在 Problem E 上的表現。

B. Main Idea of Signal Permutation and AST Comparison



對於相對簡單的測資，我們採用 Abstract Syntax Tree（AST）進行結構性比對，並不額外進行邏輯化簡或訊號拆解。為簡要說明 AST 的基本概念，我們將每個運算元（operand）或訊號（signal）視為一個節點（node），並依據邏輯運算的關係逐層延伸，最終形成一棵具備層級結構的語法樹。在比對過程中，透過對 AST 的遍歷，可有效檢查不同訊號間在結構與邏輯表達上的相似性或等價性。此方法具有良好的效率與可解釋性，特別適用於處理結構較為單純的案例。

C. Flow Chart of Programs

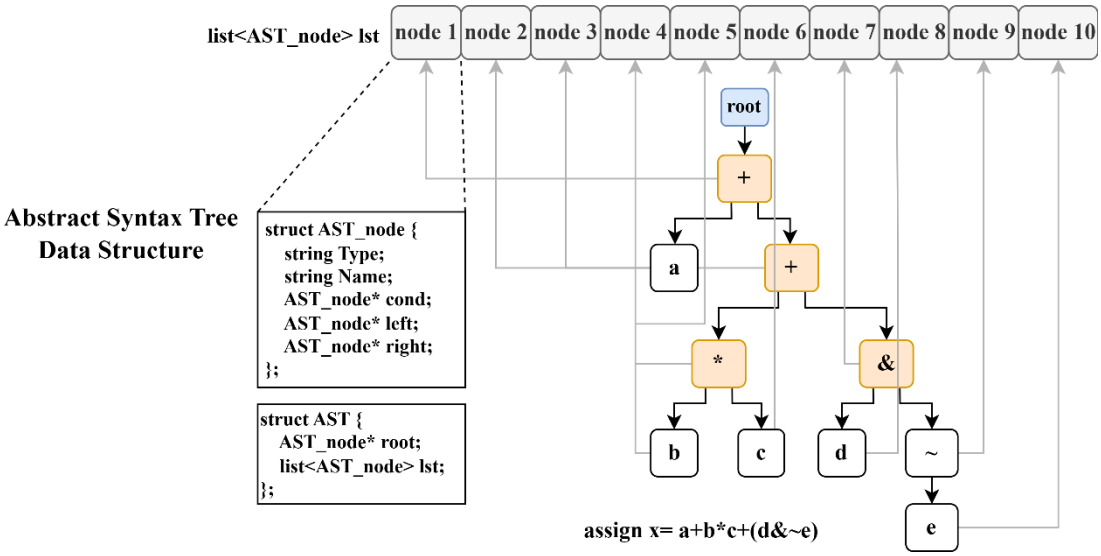


針對較簡單的測資，我們設計了一套完整的程式控制流程。首先，系統會讀入新版本的 Submodule 以及原始 Submodule，並對其訊號進行初步的解析與處理。接著根據運算子的優先順序（Operator Priority）建構對應的 Abstract Syntax Tree（AST），以還原訊號邏輯的結構。

AST 建構完成後，系統會根據訊號的類型與屬性進行分類，並依據輸入的 Update Information，優先替換掉那些已明確對應的訊號對。隨後，利用 permutation 演算法列舉出所有可能的訊號配對組合。

為提升比對準確性，所有產生的 AST 結構皆會先經過正規化（Normalization）處理，以消除語法上造成的結構差異。最終，透過 AST 結構比對，找出唯一合理且正確的訊號對應組合，並將配對結果輸出至檔案中。整體流程兼顧效率與準確性，適用於處理規模中等、結構清晰的測資情境。

D. Data Structure- Abstract Syntax Tree



我們使用 std::list 作為容器來儲存所有 Abstract Syntax Tree（AST）節點的實體。選擇 list 而非 vector 的主要原因在於記憶體重新配置（reallocation）所造成的指標失效問題。vector 在進行多次 push_back 操作時，若容量不足會自動重新配置記憶體，導致原有元素的記憶體位置改變，使得先前儲存的指標失效。為了確保每個節點在記憶體中的位置穩定，我們改以 list 儲存節點，避免上述問題。在 AST 的建構過程中，我們實際上是紀錄每個節點的指標，並透過這些指標將節點串接起來，最終形成一棵具邏輯層級結構的

Abstract Syntax Tree。此設計可維持記憶體穩定性，並支援高效的節點操作與 AST 結構管理。

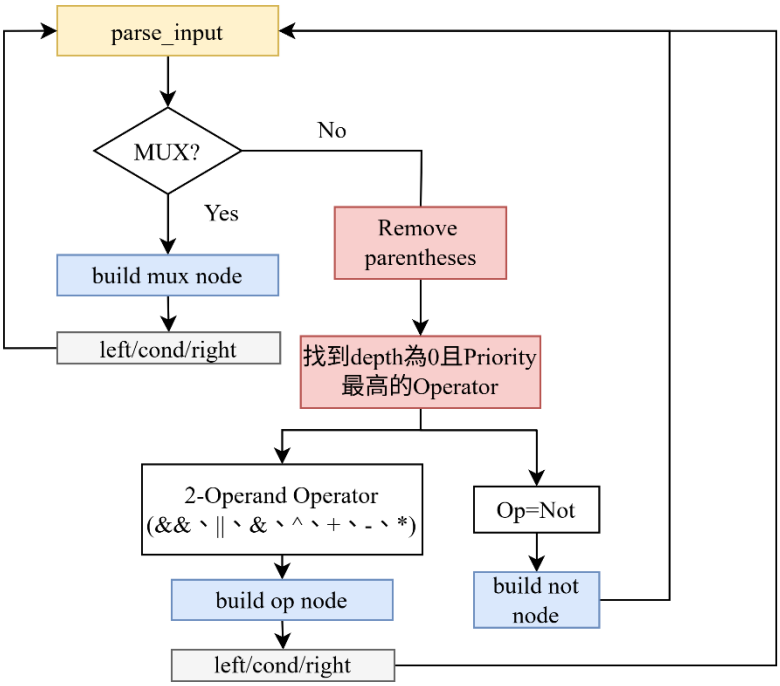
E. Algorithm- Building Abstract Syntax Tree

在建構 Abstract Syntax Tree（AST）的過程中，我們採用邊解析（parse）輸入字串、邊同步建立對應節點的策略。首先，系統會判斷輸入字串是否包含 Multiplexer（三元運算子）語法。若偵測到 MUX，我們會直接建立對應的 MUX 節點，並將字串切分為三個部分：條件式（condition expression）、true branch 與 false branch，分別對應至節點的三個子元件。

若輸入字串不屬於 MUX 結構，系統則會先檢查最外層是否包含括號，若有，則去除括號後再進行進一步分析。接著，我們會在剩餘字串中尋找「深度為 0」的運算子，亦即未被任何括號包覆的運算符號。當存在多個深度為 0 的運算子時，則根據 Verilog RTL 語法所定義的優先順序（Priority）選擇其中一個作為本層的主要運算子，並據此建立對應的 AST 節點。

在實作上，我們將運算子區分為不同類型進行處理，包括邏輯運算（Logical Operators）、比較運算（Comparison Operators）、位元運算（Bitwise Operators）、與算術運算（Arithmetic Operators）等，統稱為 ToOperate 類型的運算子。對於如 NOT 等單元運算子（Unary Operator），因其結構特性不同，亦設計獨立的處理邏輯。

每完成一層節點建構後，系統會透過遞迴方式，進一步解析與建構子字串對應的子樹節點，直至整棵 AST 完整建構完成。透過此方法，我們能夠有效還原訊號邏輯的層級與結構，為後續的比對與分析提供基礎。

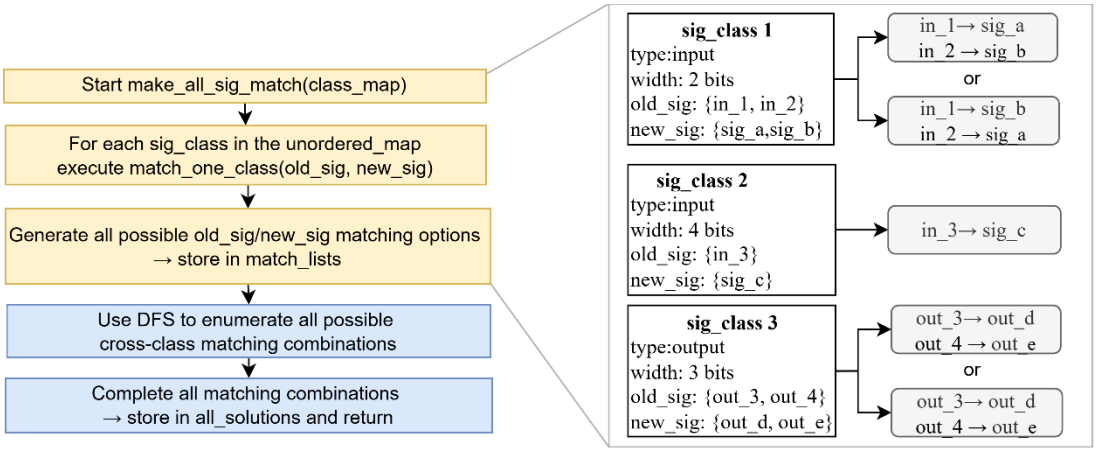


F. Algorithm- Signal Pairing Permutation

在進行訊號配對的階段中，我們採用了 permutation 演算法作為基礎策略。首先，針對每一個訊號類別（依據訊號的 I/O 型態與寬度進行分類），我們會個別產生所有可能的訊號配對組合，以涵蓋類別內的所有潛在對應關係。

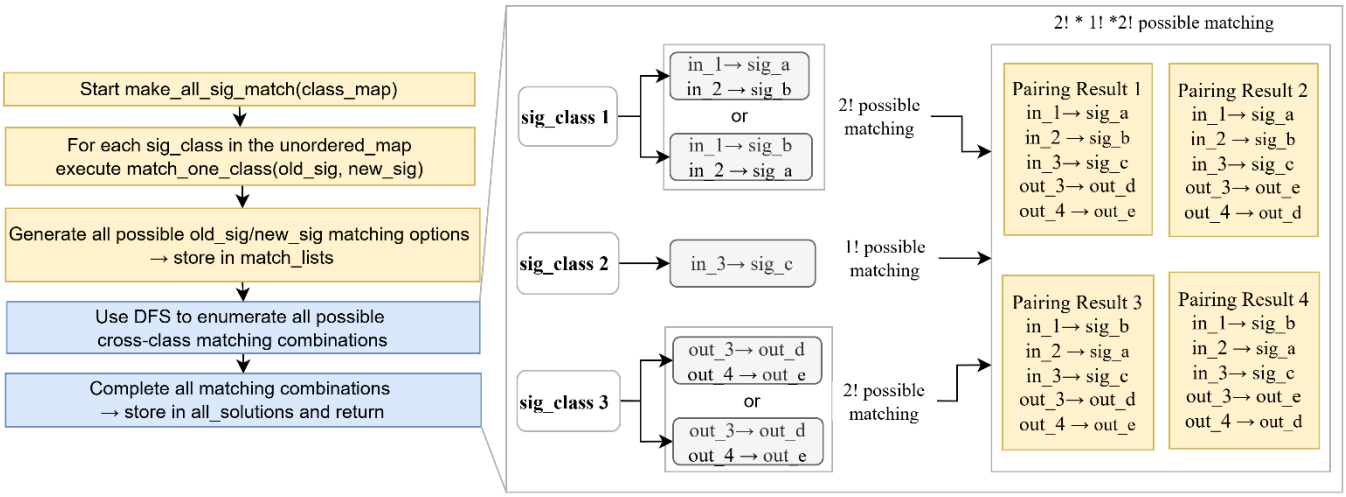
待各類別內部的配對組合生成完畢後，我們進一步用 DFS 來處理跨類別之間的訊號配對

問題。此策略有助於在大幅降低計算複雜度的同時，保留具有潛力的候選組合。經過 permutation 演算法與 DFS 雙階段處理後，我們即可獲得一個模組（Module）中所有可能的訊號配對組合。最終，我們會針對這些候選配對組合進行逐一驗證與邏輯比對，找出唯一正確且結構一致的一組訊號對應，作為後續比對分析與結果輸出的依據。



上圖展示的是「單一類別訊號配對」的處理方式，在進行實際配對之前，我們會先根據訊號特性進行分類，主要依據包括兩項：其一為訊號的方向（Input 或 Output），其二為訊號的寬度（Bit Width）。

在此分類基礎上，僅當兩個訊號的 I/O 類型相同，且寬度完全一致時，該配對才被視為合法並進一步納入考量。若任一條件不符，例如訊號方向不同或寬度不一致，則該配對將被排除，視為不可能的對應組合。此步驟可有效縮小搜尋空間，提升後續配對演算法的效率與準確性。



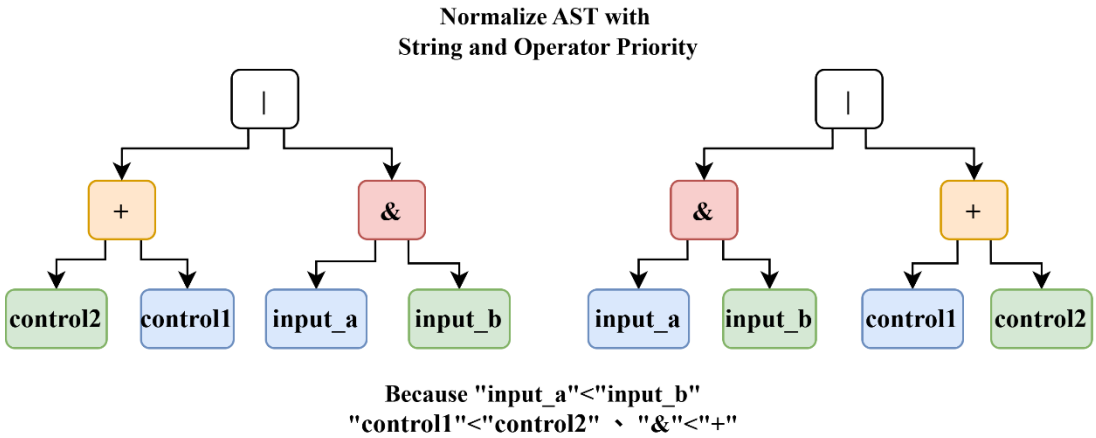
上圖展示的是在完成「單一類別訊號配對」後，進一步進行「跨類別訊號配對」的處理流程。跨類別配對的目的是將不同訊號類別（如不同方向或不同模組區塊）之間的潛在對應關係一併納入考量，從而建構出完整的配對候選集合。

唯有在跨類別訊號也成功配對之後，我們才能列舉出該模組（Module）中所有可能的訊號配對組合。因此，跨類別配對是整體訊號配對流程中不可或缺且關鍵的一環，對於後續正確性驗證與比對結果有著決定性影響。

G. Algorithm- Normalizing Abstract Syntax Tree

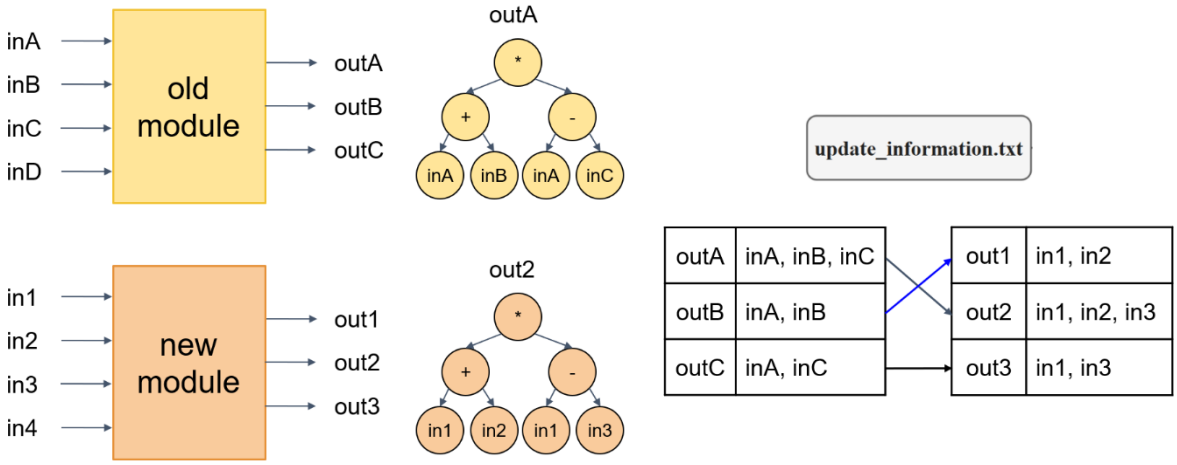
當我們取得所有可能的訊號配對組合後，下一個關鍵步驟便是從中篩選出唯一正確的一組對應關係。在此階段，我們會針對每一棵 Abstract Syntax Tree（AST）進行 Normalization（正規化）處理。此步驟至關重要，主要目的是避免因語法形式上的細

微差異，導致兩棵在邏輯上等價的 AST 被誤判為不一致。我們的正規化策略是根據運算子的優先順序（Operator Priority）以及節點中訊號名稱的字串順序，對 AST 結構進行統一排序。透過這樣的處理，可以確保邏輯等價的 AST 擁有一致的結構與表達方式，進而提升比對的準確性與可靠性。



H. Algorithm- Signal Matching

完成訊號配對後，結果會呈現訊號之間一對一的對應關係:



3. RTL to Gate-Level Netlist

A. Purposes

針對 Problem E 的 public case3，其 sub module 為 RTL 描述，而 new sub module 為 gate level netlist，因此需要將 RTL 形式的 sub module 轉換為 logic gate，以對應 new sub module 中的 port 名稱。

B. Updated Flow Chart

在 part2 d 的流程圖上，新增轉換 RTL 的功能，以特別處理 public case 3。

C. Arithmetic operations/conditional statements to logic gate netlist

RTL 有以下 8 種情況，需要考慮：

I. Conditional signals: A < B, A > B, A == B

1-bit:	n-bit: A[n-1:0], B[n-1:0]
(a) Greater than, >	(a) Greater than, >
A>B → GT = A AND (NOT B)	A>B → GT = (A[n-1] > B[n-1]) OR ((A[n-1] == B[n-1]) AND (A[n-2] > B[n-2])) OR ... ((A[1] == B[1]) AND (A[0] > B[0]))
(c) Less than, <	(c) Less than, <
A<B → LT = (NOT A) AND B	A<B → LT = (A[n-1] < B[n-1]) OR ((A[n-1] == B[n-1]) AND (A[n-2] < B[n-2])) OR ... ((A[1] == B[1]) AND (A[0] < B[0]))
(b) Equal, ==	(b) Equal, ==
A==B → EQ = A XNOR B	A==B → EQ = (A[n] == B[n]) AND ... (A[0] == B[0])

II. Bitwise concatenation: wire[2:0] = {a, b, c}

因為單純替換訊號而已，不需要額外 logic gate。

例如：把 wire[0]換成 a，

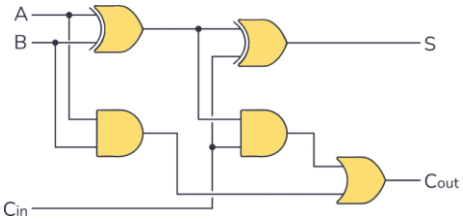
assign out = wire[0] & in[0] → and(out, a, in[0])

III. Addition: A + B

加法就是使用 full adder。

sum = A XOR B XOR Cin

Cout = (A AND B) OR (Cin AND (A XOR B))

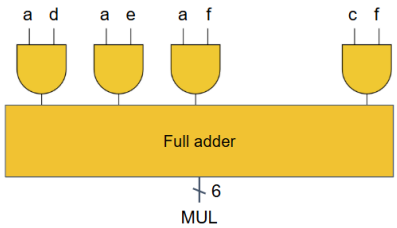


IV. Multiplication: MUL[5:0] = {a,b,c} * {d,e,f}

需對每一組位元進行 AND 運算（即 a&d, a&e, ..., c&f），再將所有 AND 結果加總。

p0 = AND(a, d), p1 = AND(a, e), p2 = AND(a, f), …, p8 = AND(c, f)

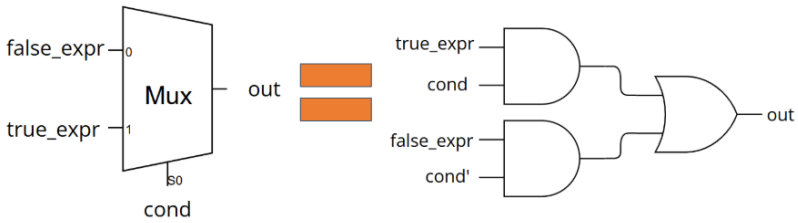
⇒ MUL[5:0] = p0 + p1 + … + p8



V. Conditional (ternary) operator: (cond) ? true_expr : false_expr

if-else 判斷語句可視為多工器（MUX），對應的邏輯轉換為：

out = (cond AND true_expr) OR ((NOT cond) AND false_expr)



VI. Bitwise shifts: in[1:0] << 1, in[1:0] >> 1

因為單純替換訊號而已，只需要考慮訊號的位數對應，不需要額外 logic gate，例如：

in[1:0] << 1 → wire[1] = in[0], wire[0]=0

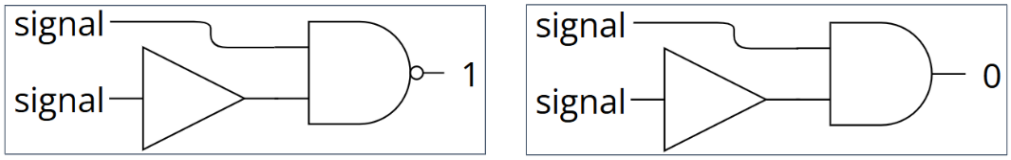
in[1:0] >> 1 → wire[1] = 0, wire[0]=in[1]

VII. Literal constants: 3'b 101, 3(10)

因為 case 沒有使用 GND 與 VDD 當 logic 0 與 logic 1，因此需要使用 logic gate 產生常數，

1 = NAND(any signal, NOT(same signal)))

0 = AND(any signal, NOT(same signal))



VIII. logic operation: &, |, ^, ~

單純轉換成 logic gate，and(out, a, b)、or(out, a, b)、xor(out, a, b)、not(inv_a, a)。

4. CNF & SAT Solver

在 Testcase02 以及 04 中，由於輸入的 Verilog 已經沒有高階抽象的語法了，所以 AST 的架構無法使用，因此我們採用了 CNF+SAT solver 的流程來進行比對

A. Conjunctive Normal Form

CNF 是一種將布林邏輯公式(Boolean formula)轉換成一連串子句(clause)之合取(AND)的標準表示形式，其中每個子句是由文字(literal)之析取(OR)組成。

形式上，一個 CNF 看起來像這樣：

$(F1 \vee \neg F2 \vee F3) \wedge (\neg F1 \vee F4) \wedge (F2 \vee F3 \vee \neg F5)$

其中：

- $\vee$ 表示 OR
- $\wedge$ 表示 AND
- $\neg$ 表示 NOT

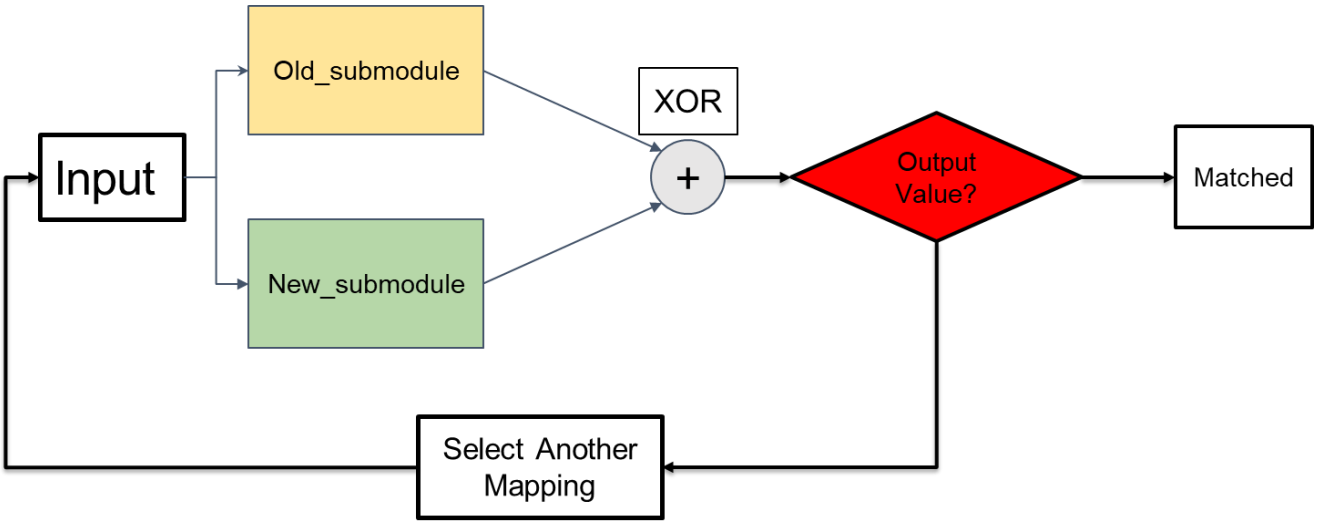
在 testcase02 以及 04 中，由於所有的宣告都是 gate-level 形式，所以我們可以很輕易地依照下列規則來轉換:

Gate		CNF
INV	(z=~a)	$(\neg a \vee \neg z) \wedge (a \vee z)$
AND	(z=a&b)	$(\neg a \vee \neg b \vee z) \wedge (a \vee \neg z) \wedge (b \vee \neg z)$
OR	(z=a|b)	$(a \vee b \vee \neg z) \wedge (\neg a \vee z) \wedge (\neg b \vee z)$
XOR	(z=a^b)	$(\neg a \vee \neg b \vee \neg z) \wedge (\neg a \vee b \vee z) \wedge (a \vee \neg b \vee z) \wedge (a \vee b \vee \neg z)$
MUX	(z=s?a:b)	$(z \leftrightarrow (s \wedge a) \vee (\neg s \wedge b))$

在將電路轉換成 CNF 形式後，我們即可使用 SAT solver 解決問題。

B. SAT Solver

SAT 的基本功能為：「找到一組 input，使得整個布林公式為真?」。在本題我們可以使用 SAT，分別對兩組 submodule 只派同一組 input，比較兩組的 output，如果相符就代表此組 Mapping 正確，記錄下來，如果不符，則在列表中選擇新的一組 mapping，重複這個過程直到找到相符的結果，如下圖。



Part3. Results on Public Testcases

以下是我們的 Program 在 Public Testcases 以及我們自行依照 Problem E 規則生成的 Testcase 上的驗證結果:

1. Public TestCase 0

public test case 0 是出現在文件上的測資，包含了簡單的 MUX、bitwise AND、bitwise OR，以及加法與減法。

```
module sub_module (
input wire [3:0] input_a,
input wire [3:0] input_b,
input wire control1,
input wire control2,
output wire [3:0] output_y
);

assign output_y=(control1 & control2)?(input_a+ input_b):
(control1 & ~control2) ? (input_a -input_b) :
(~control1 & control2) ? (input_a |input_b) :
(input_a & input_b);

endmodule
```

```
module new_sub_module (
input wire [3:0] data_x,
input wire [3:0] data_y,
input wire control1,
input wire flag_x,
output wire [3:0] result
);


assign result=(control1 & flag_x)?(data_x + data_y):
(control1 & ~flag_x)?(data_x - data_y):
(~control1 & flag_x)?(data_x | data_y):
(data_x & data_y);

endmodule
```

<u>Our Signal Matching Results: 由結果可知，訊號配對精準且正確。</u>

```
module 1 signal matching results:
new: input_a -> old: data_x
new: input_b -> old: data_y
new: output_y -> old: result
new: control1 -> old: control1
new: control2 -> old: flag_x
```

2. Public TestCase 1

相較於 Public Test Case 0，Public Test Case 1 包含較為複雜的邏輯結構，且每個 RTL 程式包含多個模組，每個模組亦具有多個輸出端:

```
module sub_module1(
    input wire  [3:0] in1,
    input wire  [3:0] in2,
    input wire        cin,
    output wire [4:0] sum);
    assign sum = 2*in1 + in2 + cin;
endmodule

module sub_module2(
    input wire  [3:0] in1,
    input wire  [3:0] in2,
    input wire  [3:0] in3,
    output wire [3:0] out1,
    output wire [3:0] out2);
    assign out1 = in1 + 3*in3*in2;
    assign out2 = (in2 > 2) ? (in1 + in3) : (in1 ^ in3);
endmodule

module sub_module3(
    input wire  [3:0] in1,
    input wire  [3:0] in2,
    input wire  [3:0] in3,
    input wire        x1,
    input wire        x2,
    input wire        x3,
    output wire [3:0] out1,
    output wire [3:0] out2
);
    assign out1 = (x1 || (~x2 && x3)) ? (in1 + in2) : (in2 + in3);
    assign out2 = (x2 && ~x3 || (x1 && ~x2 && x3)) ? (in2 + in3) : (in1 + in2);
endmodule
```

```
module new_sub_module1(
    input wire        new_in1,
    input wire  [3:0] new_in2,
    input wire  [3:0] new_in3,
    output wire [4:0] new_out1
);

    assign new_out1 = 2*new_in2 + new_in3 + new_in1;

endmodule

module new_sub_module2(
    input wire  [3:0] new_in1,
    input wire  [3:0] new_in2,
    input wire  [3:0] new_in3,
    output wire [3:0] new_out1,
    output wire [3:0] new_out2
);

    assign new_out1 = new_in2 + 3*new_in3*new_in1;
    assign new_out2 = (new_in1 > 2) ? (new_in2 + new_in3) : (new_in2 ^ new_in3);

endmodule

module new_sub_module3(
    input wire        new_in3,
    input wire        new_in5,
    input wire        new_in6,
    input wire  [3:0] new_in1,
    input wire  [3:0] new_in2,
    input wire  [3:0] new_in4,
    output wire [3:0] new_out1,
    output wire [3:0] new_out2
);

    assign new_out2 = (new_in5 || (~new_in6 && new_in3)) ? (new_in4 + new_in2) : (new_in2 + new_in1);
    assign new_out1 = (new_in6 && ~new_in3 || (new_in5 && ~new_in6 && new_in3)) ? (new_in2 + new_in1) : (new_in4 + new_in2);

endmodule
```

Our Signal Matching Results: 由結果可知，訊號配對精準且正確。

```
module 1 signal matching results:
new: sum -> old: new_out1
new: in1 -> old: new_in2
new: in2 -> old: new_in3
new: cin -> old: new_in1
```

```
module 2 signal matching results:
new: out1 -> old: new_out1
new: out2 -> old: new_out2
new: in2 -> old: new_in1
new: in1 -> old: new_in2
new: in3 -> old: new_in3
```

```
module 3 signal matching results:
new: in3 -> old: new_in1
new: in2 -> old: new_in2
new: in1 -> old: new_in4
new: out2 -> old: new_out1
new: out1 -> old: new_out2
new: x3 -> old: new_in3
new: x1 -> old: new_in5
new: x2 -> old: new_in6
```

3. Self-Generated TestCase 2

```
module datapath_unit(
    input wire  [3:0] data1,
    input wire  [3:0] data2,
    input wire  [3:0] factor,
    input wire        select,
    output wire [5:0] final_mult,
    output wire [3:0] logic_mix,
    output wire       flag
);
    assign final_mult = (select ? (factor * data1 * 2) : (factor * data2));
    assign logic_mix = data1 ^ data2 ^ factor;
    assign flag = (data2 >= 3);
endmodule

module control_logic_unit(
    input wire        cond_a,
    input wire        cond_b,
    input wire        cond_c,
    input wire  [3:0] val1,
    input wire  [3:0] val2,
    input wire  [3:0] val3,
    input wire  [3:0] val4,
    output wire [3:0] res1,
    output wire [3:0] res2,
    output wire [3:0] res3
);
    assign res1 = (cond_a && ~cond_b) ? (val1 + val2) : (val3 + val4);
    assign res2 = (~cond_c || (cond_b && cond_a)) ? (val2 + val3) : (val1 + val4);
    assign res3 = (cond_a ^ cond_c) ? (val1 + val3) : (val2 ^ val4);
endmodule
```

```
module datapath_unit_v2(
    input wire  [3:0] scale_factor,
    input wire  [3:0] input_b,
    input wire        mode_select,
    input wire  [3:0] input_a,
    output wire [5:0] scaled_result,
    output wire [3:0] xor_result,
    output wire       threshold_flag
);
    assign scaled_result = mode_select ? (2 * scale_factor*input_a ) : (input_b * scale_factor);
    assign xor_result = input_b ^ scale_factor ^ input_a;
    assign threshold_flag = (input_b >= 3);
endmodule

module control_logic_unit_v2(
    input wire  [3:0] op4,
    input wire  [3:0] op2,
    input wire  [3:0] op1,
    input wire  [3:0] op3,
    input wire        ctrl_c,
    input wire        ctrl_a,
    input wire        ctrl_b,
    output wire [3:0] out2,
    output wire [3:0] out3,
    output wire [3:0] out1
);
    assign out1 = (~ctrl_b && ctrl_a) ? (op2 + op1) : (op4 + op3);
    assign out2 = ((ctrl_a && ctrl_b) || ~ctrl_c) ? (op2 + op3) : (op4 + op1);
    assign out3 = (ctrl_c ^ ctrl_a) ? (op1 + op3) : (op2 ^ op4);
endmodule
```

TestCase2 是我們自己設計的測試案例，加入了更複雜的邏輯和更多的輸出訊號。這個測試還包含一些較複雜的交換率檢驗，用來確認我們的程式邏輯是正確的。由於測試中設計了許多陷阱，程式能順利通過，代表我們 Program 縝密且正確。

Our Signal Matching Results: 由結果可知，訊號配對精準且正確。

```
module 1 signal matching results:
new: flag -> old: threshold_flag
new: data1 -> old: scale_factor
new: data2 -> old: input_b
new: factor -> old: input_a
new: logic_mix -> old: xor_result
new: final_mult -> old: scaled_result
new: select -> old: mode select
```

```
module 2 signal matching results:
new: res1 -> old: out2
new: res3 -> old: out3
new: res2 -> old: out1
new: cond_a -> old: ctrl_c
new: cond_c -> old: ctrl_a
new: cond_b -> old: ctrl_b
new: val2 -> old: op4
new: val4 -> old: op2
new: val1 -> old: op1
new: val3 -> old: op3
```

4. Self-Generated TestCase 3

這也是我們自行生成的測試資料，包含更多的輸出訊號，並且加入了更複雜的交換率驗證以及訊號配對的測試。此外，測試中還包含了位移運算（shift operation），以進行更嚴謹的驗證。

```
module datapath_v5(
    input wire  [3:0] data_a,
    input wire  [3:0] data_b,
    input wire  [3:0] coeff,
    input wire        trigger,
    output wire [5:0] shift_result,
    output wire [3:0] and_result,
    output wire [3:0] or_result,
    output wire       valid_flag,
    output wire [3:0] shifted_twice
);
    assign shift_result   = trigger ? (data_a << 2) + (coeff >> 1) : (data_b << 1);
    assign and_result     = data_a & coeff;
    assign or_result      = data_b | coeff;
    assign valid_flag     = (trigger && (data_a != 4'b0000));
    assign shifted_twice  = coeff << 2;
endmodule

module logic_unit_v5(
    input wire        flag_a,
    input wire        flag_b,
    input wire        flag_c,
    input wire  [3:0] input_1,
    input wire  [3:0] input_2,
    input wire  [3:0] input_3,
    input wire  [3:0] input_4,
    output wire [3:0] output_1,
    output wire [3:0] output_2,
    output wire [3:0] output_3,
    output wire [3:0] output_4,
    output wire [3:0] output_5
);
    assign output_1 = (flag_a && flag_b) ? (input_1 + input_2) : (input_3 + input_4);
    assign output_2 = flag_c ? (input_2 << 1) : (input_3 >> 1);
    assign output_3 = (flag_b && ~flag_c) ? (input_4 + input_1) : (input_2 + input_3);
    assign output_4 = input_1 ^ input_2 ^ input_3;
    assign output_5 = input_4 << 2;
endmodule
```

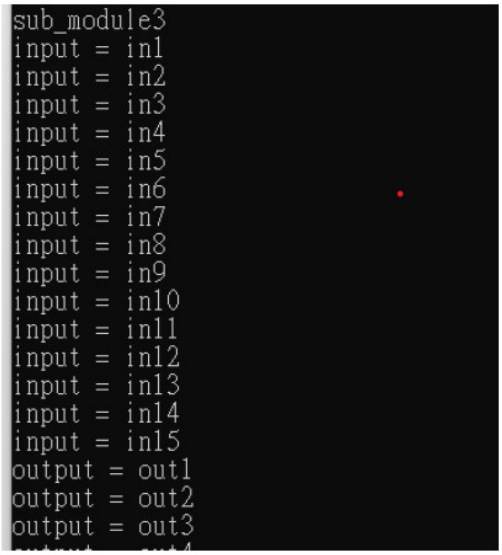
```
module datapath_v5(
    input wire  [3:0] in_data2,
    input wire  [3:0] in_data1,
    input wire  [3:0] in_coeff,
    input wire        ctrl_trigger,
    output wire [5:0] out_shifted_sum,
    output wire [3:0] out_bitwise_and,
    output wire [3:0] out_bitwise_or,
    output wire       out_valid,
    output wire [3:0] out_coeff_shift2
);
    assign out_shifted_sum  = ctrl_trigger ? (in_coeff >> 1)+(in_data1 << 2) : (in_data2 << 1);
    assign out_bitwise_and  = in_data1 & in_coeff;
    assign out_bitwise_or   = in_coeff | in_data2;
    assign out_valid        = ((in_data1 != 4'b0000)&&ctrl_trigger );
    assign out_coeff_shift2 = in_coeff << 2;
endmodule

module logic_unit_v5(
    input wire        ctrl_b,
    input wire        ctrl_a,
    input wire        ctrl_c,
    input wire  [3:0] in1,
    input wire  [3:0] in2,
    input wire  [3:0] in3,
    input wire  [3:0] in4,
    output wire [3:0] out1,
    output wire [3:0] out3,
    output wire [3:0] out2,
    output wire [3:0] out5,
    output wire [3:0] out4
);
    assign out1 = (ctrl_b && ctrl_a) ? (in1 + in2) : (in4 + in3);
    assign out2 = ctrl_c ? (in2 << 1) : (in3 >> 1);
    assign out3 = (ctrl_b && ~ctrl_c) ? (in1 + in4) : (in2 + in3);
    assign out4 = in1 ^ in2 ^ in3;
    assign out5 = in4 << 2;
endmodule
```

<u>Our Signal Matching Results: 由結果可知，訊號配對精準且正確。</u>

```
module 1 signal matching results:
new: valid_flag -> old: out_valid
new: data_a -> old: in_data2
new: data_b -> old: in_data1
new: coeff -> old: in_coeff
new: and_result -> old: out_bitwise_and
new: or_result -> old: out_bitwise_or
new: shifted_twice -> old: out_coeff_shift2
new: shift_result -> old: out_shifted_sum
new: trigger -> old: ctrl_trigger
```

```
module 2 signal matching results:
new: output_1 -> old: out1
new: output_2 -> old: out3
new: output_3 -> old: out2
new: output_4 -> old: out5
new: output_5 -> old: out4
new: flag_a -> old: ctrl_b
new: flag_b -> old: ctrl_a
new: flag_c -> old: ctrl_c
new: input_1 -> old: in1
new: input_2 -> old: in2
new: input_3 -> old: in3
new: input_4 -> old: in4
```

5. Public case2 and 4 parser



The output will be the name of input, output and wire

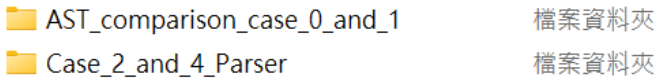
我們的 parser 可以把 sub module 裡的 input output wire 存進 vector，以方便後面進行訊號匹配。

6. Conclusion

由以上測試可知，我們以 Abstract Syntax Tree 搭配 Permutation Algorithm，實作 Signal Matching 後，通過 <u>Public Case 0、Public Case 1、以及 Self-Generated Testcases</u>:，可以處理不涉及邏輯化簡且沒有額外定義內部訊號的所有 Combinational Verilog Module

Part 4. Compilation and Execution

首先解壓縮我們的檔案，會有兩個資料夾



1. AST_comparison_case_0_1

這個部分是 Part 2-b 所提到，針對 simple case，不細部拆分邏輯，單純以運算元的交換率去正規化 AST，並以 Permutation Algorithm 做訊號配對，按照以下方式編譯並執行:

i. **compile command: g++ CAD.cpp AST.cpp -o CAD**

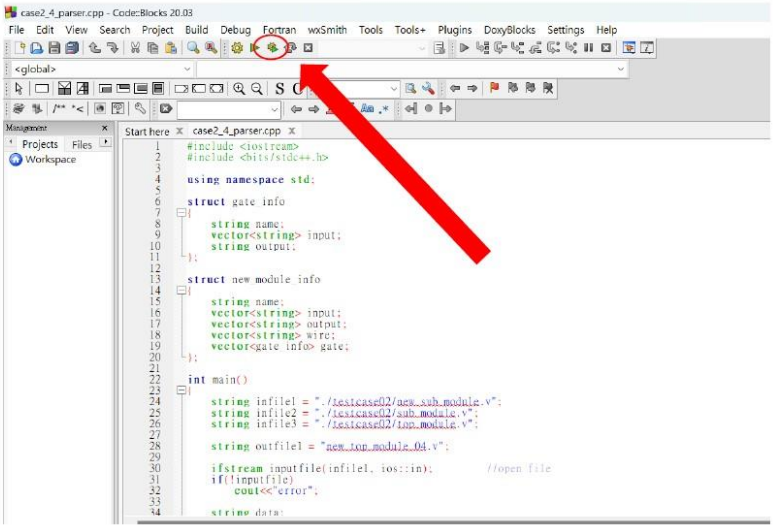
ii. **execution(把 testcase 的所有 file 放到根目錄並執行以下 command):**

./CAD top_module.v sub_module_0.v new_sub_module_0.v update_information new_top_module.v
./CAD top_module.v sub_module_1.v new_sub_module_1.v update_information new_top_module.v
./CAD top_module.v sub_module_2.v new_sub_module_2.v update_information new_top_module.v
./CAD top_module.v sub_module_3.v new_sub_module_3.v update_information new_top_module.v
./CAD top_module.v sub_module_4.v new_sub_module_4.v update_information new_top_module.v
./CAD top_module.v sub_module_5.v new_sub_module_5.v update_information new_top_module.v

2. Case_2_and_4_Parser

這個部分是專門針對 TestCase2 與 TestCase4 所設計的 Pulsar 模組，主要功能是解析更複雜的 Gate-level 結構，並進行相應的轉換與訊號替換，最終會產生一個輸出的結果檔。

Parser of case 2 and 4



Compile the program:
g++ -std=c++11 program_name.cpp -o verilog_updater

Run the executable:
./verilog_updater

Part 5. Conclusions

以下是我們在 Check Point 3 的主要進度與成果:

a. **以 Abstract Syntax Tree 搭配 Permutation Algorithm，實作 Signal Matching，通過 <u>Public Case 0、Public Case 1、以及 Self-Generated Testcases</u>**: 不涉及邏輯化簡且沒有額外定義內部訊號的所有 Combinational Verilog Module ，詳細 demo 成果如前面所述。

b. 針對 Verilog Combinational Logic 的所有 Operator，依照 Operator Priority 做訊號的拆分並實作建構 Abstract Syntax Tree 的 Program。

c. 針對較複雜、涉及複雜邏輯驗證與比對的 <u>Public Case 2、Public Case 4</u> 實作 Parser，完成訊號比對與替換，並於影片中演示。

d. 針對較複雜、涉及複雜邏輯驗證與比對的 <u>Public Case 2、Public Case 4</u> **嘗試開源的 SAT Solver 演算法，在 Toy Case 上完成驗證**

Suggestion

您好，非常感謝助教用心地幫我們評分 Final Project，也謝謝您在我們遇到問題或有疑問時，總是能夠迅速而清楚地回覆，這點真的讓我們受益良多，深感感激。目前對於 Final Project 的安排與進行方式沒有太多建議，整體上覺得非常完善，幾乎無可挑剔。最後，祝福助教在未來的研究與求職路上都能一切順利，事事如意！